

第二章

CARLA环境搭建与实操讲解

主 讲：阿勇

公众号：自动驾驶之心

主要内容

1 CARLA的安装与启动

2 实战案例运行

3 CARLA基础API解析

4 CARLA上手实战

5 CARLA知识点梳理

34

CARLA的安装与启动

安装环境

系统要求:

- Ubuntu 16.04/18.04. Ubuntu 20.04也可以Build Carla, 但不支持Autoware, 所以不建议使用该版本。
- Nvidia驱动提前安装好, 并且电脑配置一个至少4G的GPU.
- 硬盘里至少预存100G空间

下载方式: <https://github.com/carla-simulator/carla/releases>

版本: 0.9.10/0.9.10.1

解压: `tar -xzvf CARLA_0.9.10.1.tar.gz`

CARLA 0.9.10.1 (development)

Patch Release 0.9.10.1

- [Ubuntu] [CARLA_0.9.10.1.tar.gz](#)
- [Ubuntu] [AdditionalMaps_0.9.10.1.tar.gz](#)
- [Ubuntu] [CARLA_0.9.10.1_RSS.tar.gz](#)
- [Windows] [CARLA_0.9.10.1.zip](#)
- [Windows] [AdditionalMaps_0.9.10.1.zip](#)

自动驾驶之心



初识界面

启动命令： ./CarlaUE4.sh -prefernvidia

如果性能比较差： ./CarlaUE4.sh -quality-level=Low

相关的启动配置：

carla-rpc-port=N: 侦听端口 N 处的客户端连接。默认情况下，流式端口 Streaming port 设置为 N+1;

carla-streaming-port=N: 指定用于传感器数据流的端口。使用 0 获取随机未使用的端口。第二个端口将自动设置为 N + 1;

quality-level={Low,Epic}: 更改图形质量级别;

-carla-server: 让 carla以服务的方式运行;

-benchmark -fps=15: 引擎以1/15秒的固定时间逐步运行;

-windowed -ResX=800 -ResY=600: 屏幕窗口大小;

视角按键说明：

Q: 向下

E: 向上

W: 向视角前方

S: 向视角后方

A: 向视角左方

D: 向视角右方



导入环境

使用python2! (因为后面的ros-bridge只支持python2, 但是算法可以使用python3, 是独立的)

```
sudo apt-get install libomp5
sudo apt-get install libpng16-16
sudo apt-get install aria2
pip2 install pygame==1.9.6 -i https://pypi.tuna.tsinghua.edu.cn/simple
```

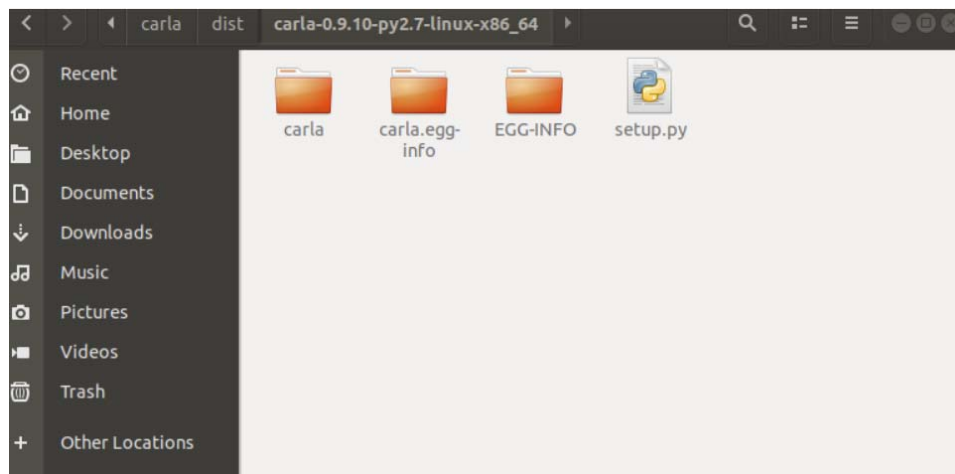
为了能在python中import carla成功, 在bashrc中添加
export PYTHONPATH=\$PYTHONPATH:~/CARLA_0.9.10.1/PythonAPI/carla/dist/carla-0.9.9-py2.7-linux-x86_64.egg
export CARLA_ROOT=/home/xx/carla_0.9.10.1 **xx是自己的用户名**
这种方法可以保证在终端导入成功, 如果使用类似Pycharm等IDE如果还是不成功, 可以采用以下方法:

```
cd ~/carla/PythonAPI/carla/dist
unzip carla-0.9.10-py2.7-linux-x86_64.egg -d carla-0.9.10-py2.7-linux-x86_64
cd carla-0.9.10-py2.7-linux-x86_64,新建一个空白的setup.py
```



导入环境

www.zdjszx.com



```
from distutils.core import setup
setup(name='carla',
      version='0.9.10.1',
      py_modules=['carla'],
      )
```

接着

```
pip2 install -e ~/.carla/PythonAPI/carla/dist/carla-0.9.10-py2.7-linux-86_64
```

完成导入

自动驾驶之心





35

实战案例运行

动手控制汽车

```
cd Carla_0.9.10.1/PythonAPI/examples/
```

```
ls
```

```
python manual_control.py
```

Q: 倒车

空格: 手刹

P: 开启/关闭自动驾驶

M: 切换到手动模式

,: 减档

.: 加档

CTRL + W : 同时按下 CTRL + W , 在放开 CTRL + W , 车会一直以 60 km/h 的速度前进

L : 控制车灯切换。雾灯、近光灯等切换。

SHIFT + L : 切换远光灯

Z/X : 转向灯。Z 左转向, X 右转向。

I : 车内照明灯。

TAB : 切换视角

数字1-9或者N: 切换不同类型的 camera 和 lidar

G : 切换到毫米波雷达

V : 选地图图层 (Shift+V , 地图有多个图层, 切换顺序和V 相反)

R : 时时记录车辆走行情况

CTRL + P : 回放R的记录

F1 : 显示/不显示页面左侧和sensor相关的一些信息, 例如加速度, 陀螺仪, GNSS等

H/? : H和? 可以弹出帮助命令

ESC : 退出pygame

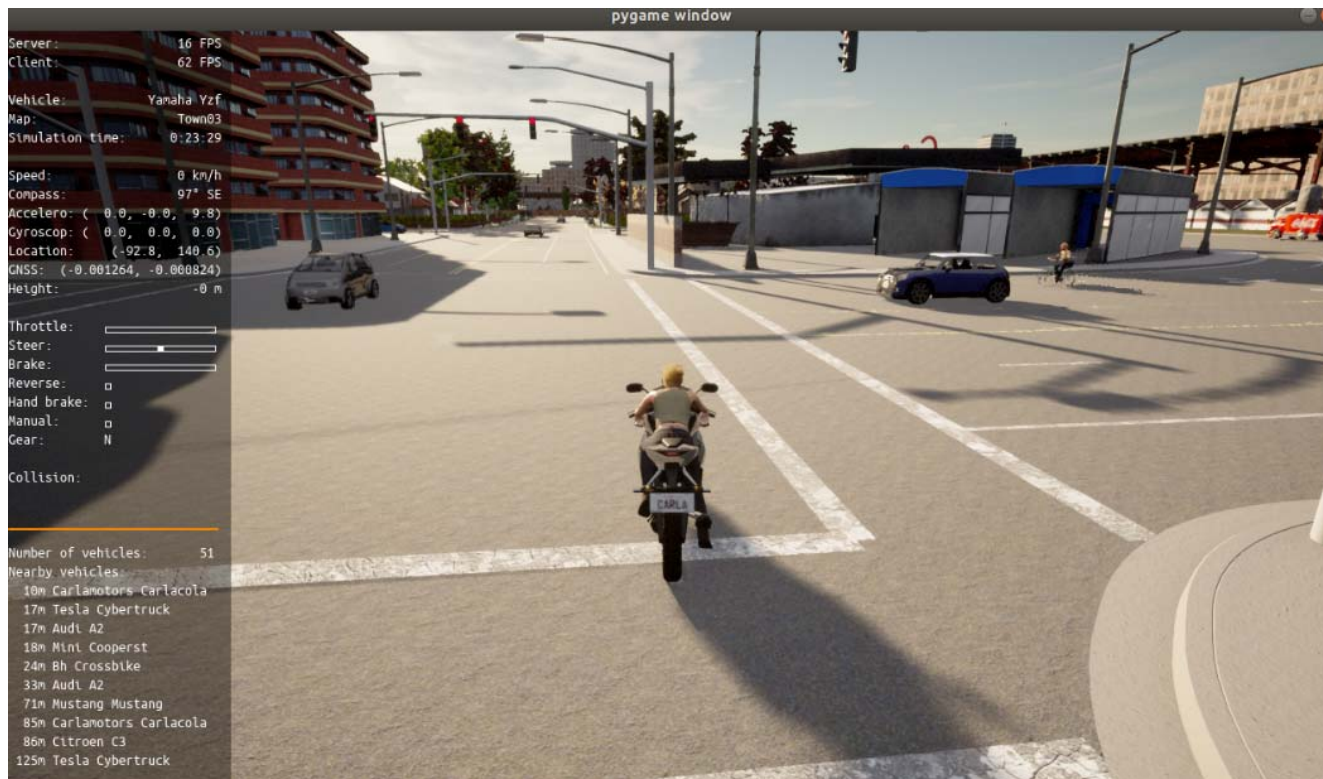
自动驾驶之心



生成交通环境

```
cd Carla_0.9.10.1/PythonAPI/examples/  
ls
```

python spawn_npc.py -n 50 就会随机生成50个交通参与者，在此基础上，再打开一个终端，运行手动控制汽车程序，即可与之交互



自动驾驶之心





36

Carla基础API解析

世界建立与观察者生成

client 是用户同整个拟真环境进行信息交互的一个模块，用户对于拟真环境的控制基本都要依托于它。Client的运行需要一个IP地址和一个端口来和server进行通信，在一个server上同时可以有多个client运行。

```
./CarlaUE4.sh -world-port=2023 #启动carla的时候设置端口，默认2000
```

```
client = carla.Client('localhost', 2023)#用户通过Client载体与python API与仿真环境交互，所以我们第一步就是要创建Client，默认2000是端口，  
client.set_timeout(2.0)#设置一个timeout时间防止连接时间过久,2.0是秒数。  
client.load_world('Town01')#更改地图
```

world是拟真环境。world包含了很多重要的方法比如说生成车辆行人等物品，改变天气，获得环境中各种车辆的信息等。与可以多个同时运行的client不同，每次模拟中只会存在一个world，并且在通过client更换地图时，这个world将被销毁并由新的代表新地图的world代替。获得world之后，我们就可以通过它去访问其他在模拟中的物品，例如天气，观察者，地图等

```
world = client.get_world()#构建的Client来获取仿真世界（World）
```



世界建立与观察者生成

Actor 在Carla中是所有可以变化影响其他Actor并且对其他Actor作出反应的元素，包括：传感器(sensor), 交通标志和信号灯(Traffic lights and signs), 车辆(Vehicle), 行人(Walker)。

Blueprints: Blueprints是创建生成Actor所需要的数据，包括模型，模型对应动画等。

```
#如果想生成一个Actor，必须先定义它的蓝图(Blueprint)，这就好比造东西前要先设计图。
```

```
blueprint_library = world.get_blueprint_library()#拿到所有物体的蓝图
```

```
#可以对蓝图进行进一步的筛选
```

```
vehicle_blueprints = world.get_blueprint_library().filter('*vehicle*')#获得车辆的蓝图
```

```
ped_blueprints = world.get_blueprint_library().filter('*pedestrian*')#获得行人的蓝图
```

```
ego_vehicle_bp = vehicle_blueprints.find('vehicle.mercedes-benz.coupe')#找到奔驰的蓝图
```

```
ego_vehicle_bp.set_attribute('color', '0, 0, 200')#给车加上特定的颜色
```

```
ego_vehicle_bp.set_attribute('role_name', 'hero')# 设置主车蓝图的属性中的角色名
```

构建好蓝图以后，下一步便是选定它的**出生点**。我们可以给固定的位子，也可以赋予随机的位置，不过这个位置必须是空的位置。



世界建立与观察者生成

```
transform = random.choice(world.get_map().get_spawn_points())#找到所有可以作为初始点的位置并随机选择一个
ego_vehicle = world.spawn_actor(ego_vehicle_bp, transform)#在这个位置生成汽车
```

汽车生成以后，我们便可以随意挪动它的初始位置，定义它的动态参数。

```
location = ego_vehicle.get_location()
location.x += 10.0
ego_vehicle.set_location(location)

# 把它设置成自动驾驶模式
ego_vehicle.set_autopilot(True)
# actor.set_simulate_physics(False)#在中途将这辆车“冻住”，通过抹杀它的物理仿真
```

使用完成后要记得销毁，否则会内存泄漏

```
#当这个脚本运行完后要将这个汽车销毁掉，否则它会一直存在于仿真世界，可能影响其他脚本的运行。
ego_vehicle.destroy()# 如果注销单个Actor
client.apply_batch([carla.command.DestroyActor(x) for x in actor_list])# 如果你有多个Actor 存在list里，想一起销毁。
```

自动驾驶之心



世界建立与观察者生成

在此基础上，考虑批量生成车辆，主要思路是批量生成出生点（避免相撞），蓝图，设置自动驾驶、销毁

```
# 创建车辆批量操作列表
number_of_vehicles = 50
batch = []
for i in range(min(number_of_vehicles, len(spawn_points))):
    spawn_point = random.choice(spawn_points) # 随机选择生成点
    vehicle_bp = random.choice(vehicle_blueprints) # 随机选择一辆车的蓝图
    batch.append(carla.command.SpawnActor(vehicle_bp, spawn_point))
    spawn_points.remove(spawn_point)

# 执行批量操作
vehicles_id_list = []
for i, response in enumerate(client.apply_batch_sync(batch, True)):
    if response.error:
        logging.error(response.error)
    else:
        print("Spawned Actor", response.actor_id)
        vehicles_id_list.append(response.actor_id)
        # 设置车辆自动驾驶
        vehicle = world.get_actor(response.actor_id)
        vehicle.set_autopilot(True, traffic_manager.get_port())
```



世界建立与观察者生成

在此基础上，考虑批量生成行人，行人与车辆不同的地方在于行人的行动需要有walk控制器来完成，此外行人可以设置速度、和横穿马路的参数

```
#创建行人批量操作列表
number_of_pedestrian = 25
ped_spawn_points = []
controllers = [] # 用于存储所有的行人控制器
spawned_locs = set() # 创建一个空集合来跟踪位置
for _ in range(number_of_pedestrian):
    spawn_point = carla.Transform()
    loc = world.get_random_location_from_navigation()
    if loc != None and loc not in spawned_locs:#avoid adding repeat ele
        spawn_point.location = loc
        spawned_locs.add(loc)
        # 在地图上随机生成num_walkers个行人，每个行人为行人蓝图库中的随机行人
        walker_bp = random.choice(ped_blueprints)
        if walker_bp.has_attribute('is_invincible'):#取消行人无敌状态
            walker_bp.set_attribute('is_invincible', 'false')
        ped = world.try_spawn_actor(walker_bp, spawn_point)#生成行人
        ped_spawn_points.append(ped)
```



世界建立与观察者生成

在此基础上，考虑批量生成行人，行人与车辆不同的地方在于行人的行动需要有walk控制器来完成，此外行人可以设置速度、和横穿马路的参数

```
# 创建walker控制器
controller_bp = blueprint_library.find('controller.ai.walker')
# 为整批行人各自生成对应的控制器，并把控制器添加到代表批量控制器的列表中
for walker in world.get_actors().filter('*pedestrian*'):
    controllers.append(world.try_spawn_actor(controller_bp, carla.Transform(), walker))
# 批量启动行人控制器，并设置控制器参数
for i in range(len(controllers)):
    destination = world.get_random_location_from_navigation()# 设置行人的行为，比如目的地和速度

    speed = random.choice([1.5, 3.0]) # 走路或跑步的速度
    if destination:
        controllers[i].start()
        controllers[i].go_to_location(destination)
        controllers[i].set_max_speed(speed)

# 设置行人横穿马路的参数
world.set_pedestrians_cross_factor(10)
```



世界建立与观察者生成

观察者视角 地图加载好后，视角会在观察者的默认位置，可以通过W A S D Q E来移动视角，也可以在代码中设置观察者视角当我们去观察仿真界面时，我们会发现视野并不会随车一起移动，所以经常会跟丢它。解决这个问题的办法就是把spectator对准汽车

```
spectator = world.get_spectator()#获得世界的观察者
transform = spectator.get_transform()# 获得观察者的方位信息
#可以在此基础上设置新的方位
location = transform.location + carla.Location(x=-30,z=20)
rotation = carla.Rotation(pitch=-20, yaw=-20, roll=0)
new_transform = carla.Transform(location, rotation)
spectator.set_transform(new_transform)#完成设置
#还可以获取车辆视角
transform = ego_vehicle.get_transform()
spectator.set_transform(carla.Transform(transform.location + carla.Location(z=20),#将观察视角
安装在车辆高20，角度俯视
carla.Rotation(pitch=-90)))
```

上面的代码能够固定视角，但是只在一开始固定了，导致车一移动就又跟丢了，正确的做法是在循环中固定视角，保证跟踪车辆。

完成作业：在carla中批量生成车辆与行人，并且时刻跟踪主车视角

自动驾驶之心



同步/异步模式

CARLA仿真器提供了两种模式来控制 and 运行仿真：**异步模式**和**同步模式**。这两种模式控制着数据的流动和时间的流逝，对于不同的仿真需求和场景可能会选择不同的模式。同步与异步的正确设置与否将直接决定最终仿真时的结果的效率与正确率，官网介绍

https://carla.readthedocs.io/en/latest/adv_synchrony_timestep/#setting-synchronous-mode

异步模式 (Asynchronous Mode) 是CARLA的默认设置，模拟的时间会根据真实时间自动推进，不需要用户干预。在这种模式下：

- 模拟器会尽可能快地运行，不等待客户端的输入或命令。
- 每次模拟步骤完成后，物理和渲染更新会立即进行。
- 客户端（例如脚本或用户应用程序）可以以任何它们想要的频率运行和发送命令。
- 异步模式可能导致不确定的时间步长，因为模拟的速度会受到计算机性能和网络延迟的影响。异步模式适用于对实时性要求高的应用场景，比如**交互式驾驶模拟或实时演示**。

同步模式下，仿真时间的推进由用户控制，这允许精确的时间步长控制和更可预测的结果。在这种模式下：

- 用户（通常是客户端脚本）需要显式地告知仿真器进行下一个时间步长。
- 仿真器会等待客户端的信号后，才更新物理和渲染。
- 所有的传感器数据和仿真结果会在每个时间步长同步返回给客户端，确保数据的一致性。
- 同步模式通常用于需要**精确重现**和控制的场景，如科学研究、数据收集或系统测试。
- 同时运行多个client时，**只能有一个client开启同步模式**，因为server会对每个收到的“ready to go”信息进行反应，多个client开启同步模式将会发送过多“ready to go”信息导致同步失败同步模式提供了对仿真的精确控制，但需要更复杂的客户端逻辑来管理时间步长的推进。

设置同步模式 在同步和异步模式之间切换只需要改变 **settings.synchronous_mode** 的值即可，如果启用了同步模式，并且正在运行Traffic Manager，则Traffic Manager也必须设置为同步模式。

自动驾驶之心



同步/异步模式

时间步长 Carla模拟世界中的时间和现实世界中的时间是不同的，模拟世界有着自己的时钟和时间，由服务器控制，两个临近的模拟时刻之间模拟世界经过的时间我们称之为**模拟时间步长**。服务器计算两个模拟时刻需要一些现实时间，计算两个模拟时刻所需的现实时间我们称之为**现实时间步长**。现实时间步长可能会与模拟时间步长时长不同。

现实步长由服务器的性能以及场景的复杂程度决定，而模拟时间步长可以由使用者自行设定。服务器可能需要几毫秒来计算两个模拟场景，而这两个模拟场景之间可能模拟世界已经经过了1秒，也就是说，模拟时间步长为1s，我们可以根据我们的需求来设置为固定模拟时间步长或者可变模拟时间步长。

可变时间步长是CARLA的默认模式,模拟步骤之间经过的时间将是服务器计算所需时间。

当我们使用可变时间步长设定时，回放模拟就会变得复杂一些：如果服务器使用可变时间步长运行，那么时间步长将与原始模拟不同，对于缺失的步长的数据将使用记录的数据进行插值，这可能会**导致回放模拟结果与原始记录结果不同**。如果服务器被强制复现完全相同的时间步长，尽管回放模拟的时间步长是相同的，但是由于服务器的性能波动或者差异，真实世界时间将会不同，模拟将以奇怪的时间波动方式回放。同时，使用可变时间步长也会导致浮点数算术误差，因为时间是连续变量，每个时间步长中间对于时间的浮点误差累积可能会导致最终结果误差甚至错误，以至于无法重现回放模拟。

```
settings = world.get_settings()
settings.fixed_delta_seconds = None # 设置可变时间步长
world.apply_settings(settings)
```



同步/异步模式

固定时间步长每个模拟步骤之间经过的时间保持恒定。如果将其设置为0.5秒，每秒将有两个模拟帧。使用相同的时间增量对于从模拟中获取数据是最好的方式。**当我们使用固定时间步长设定时进行回放模拟将很容易**，将服务器的时间步长设定被记录模拟的时间步长即可。

```
settings = world.get_settings()
settings.fixed_delta_seconds = 0.05
world.apply_settings(settings)
```

在进行仿真项目时，选择正确的模式对于实现项目目标至关重要：

- 如果你需要快速且流畅的仿真体验，并且对时间步长的精确度要求不高，**异步模式**可能是更好的选择。
- 如果你需要进行精确的数据收集、重放或任何需要严格时间控制的测试，**同步模式**将是必需的。
- 异步模式 + 可变时间步长：这是CARLA的默认状态。Server和client是异步的。模拟时间会自动根据现实时间调整。注意进行回放模拟时需要考虑浮点数算术误差和server之间可能存在的时间步长差异。
- 异步模式 + 固定时间步长：Server将尽可能快地运行。检索到的信息将与模拟中的模拟时间被一一对应起来，所以回放模拟时不会产生浮点数算术误差。如果服务器性能足够快，这种配置可以在较少的实际时间内模拟较长的模拟时间段。
- 同步模式 + 固定时间步长：Client将控制模拟进程。直到client发送信息后Server才会计算下一步。当同步性和精确性很重要时，这是**最佳模式**。特别是在client速度较慢或有多个传感器读取信息时。**在你设置了同步模式的client完成了它的任务准备停止/销毁时，千万别忘了将世界设置回异步模式，否则server会因为找不到它的同步客户而卡死。**



分析同步模式的代码

交通流生成

Traffic Manager 简称TM，是仿真里用来控制NPC车辆行为的模块。它负责处理车辆的自动驾驶功能，包括路径规划、避免碰撞、遵守交通规则等。Traffic Manager 旨在提供一个尽可能逼真的交通模拟，包括车辆的各种行驶和交互行为。另外，在同步模式下，设置AutoPilot的车辆必须依附于设置为同步模式的traffic manager才能跑起来。

数据流：

- **第一阶段：Vehicle Registry.** 首先Server端的Lifecycle管理器会扫描simulation里所有的汽车、行人的信息，删除掉已经died的物体，然后Traffic Manager会把部分车辆注册到自己旗下（至于哪些会被选中是由用户决定的），这些被选中的车辆的行为就会被该traffic manager控制（可以有多个Traffic manager同时存在，后面会详细讲述）
- **第二阶段：Localization Stage.** In Memory Map这个外部模块会缓存整张地图的拓扑结构以及每辆汽车的短期路线规划（实质上是一系列的waypoint），TM在这一阶段做的就是读取in memory map，把自己管辖的车辆的位置、路径规划存到内部模块path buffer & vehicle tracking(PBVT)里面，这样方便后面的函数快速便捷读取相关信息。
- **第三阶段：Collision Stage.** 这一阶段TM会根据每个管辖车辆的周边环境和他们的路线规划来判断是否有潜在车祸可能，如果有的话会将此信息传递给motion planner阶段。
- **第四阶段：Traffic Light Stage.** 与上一阶段类似，TM会探测每一辆车是否接近红灯或路口，将信息传给下一阶段。
- **第五阶段：Motion Planner.** 将上个两阶段的信息整合到一起，制定每一辆所管辖的车下一步的举动（比如变道、急刹或者正常行驶），设立下一个目标点，将该点传给PID控制器，得到方向盘应该转动的角度和踩油门的力度、刹车的力度。
- **第六阶段：Send Command Array.** 将上述每一辆车的控制命令一次性分发到每辆车上，实现控制。

自动驾驶之心



交通流生成

官方文档总结的TM的API效果

项目	描述
常规	- 创建连接到端口的 TM 实例 - 检索连接 TM 的端口。
安全条件	- 设置最小跟车距离（停车时） - 设定速度百分比（相对于限速，例如限速60，百分比设置为70，车速就为42） - 重置交通信号灯
碰撞管理	- 启用/禁用特定车辆之间的碰撞。 - 使车辆忽略所有其他车辆。 - 使车辆忽略所有行人。 - 使车辆忽略所有交通信号灯。
变道	- 强制变道，忽略可能的碰撞。 - 启用/禁用特定车辆的变道行为。
混合物理模式	- 启用/禁用混合物理模式。 - 更改启用物理运算的半径。

```
traffic_manager.set_global_distance_to_leading_vehicle(2.0) # 设置前车距离
traffic_manager.global_percentage_speed_difference(80) # 原来速度的80%
traffic_manager.set_hybrid_physics_mode(True) # 附近一定范围的车辆会开启物理特性。所谓的物理特性就是在运动时会考虑一些真实的物理限制，比如轮胎摩擦、道路曲率等混合物理模式就是为了减少计算消耗，只让ego-vehicle附近的交通更具有真实性，注意只有在存在主车（即有车辆的属性被设置为hero时）混合物理模式才能生效
blueprint.set_attribute('role_name', 'hero' )
# 设置种子，以便必要时行为可以重复
traffic_manager.set_random_device_seed(0)
random.seed(0)
```

分析案例代码，同步模式下批量生成指定数量的汽车并设为自动驾驶，任务完成后，批量销毁

自动驾驶之心



常见传感器配置-传感器简介

Carla中的传感器可以大致分为4类，分别为Camera, lidar, detector, other

camera会在每次模拟环境更新 world.tick() 时以自身视角对模拟环境拍照进行记录，大部分camera会输出一个 图像

传感器	蓝图	输出	功能
Depth camera	sensor.camera.depth	carla.Image	编码每帧每个像素到相机的距离，创建深度图，使用灰度图保存到磁盘，精度是毫米级
RGB camera	sensor.camera.rgb	carla.Image	最常用的车载相机，输出一张清晰的与肉眼所见相似的图像
Optical Flow		carla.Image	将画面中每个图像的移动渲染出来
DVS camera	sensor.camera.dvs	carla.DVSEventArray	动态视觉传感器，作为事件流异步测量亮度强度的变化
Semantic segmentation camera	sensor.camera.semantic_segmentation	carla.Image	语义分割相机，将不同种类物体以不同颜色显示
Instance segmentation sensor	sensor.camera.instance_segmentation	carla.Image	实例分割相机，将不同tag和不同id的物体以不同颜色显示

自动驾驶之心



常见传感器配置-传感器简介

与camera不同，detector不会在每一次模拟环境更新时输出信息，detector只在它所绑定的actor（一般为vehicle）触发特定事件时才会获取信息。

传感器	蓝图	输出	功能
Collision detector	sensor.other.collusion	carla.CollisionEvent per collision	记录碰撞事件
Lane invasion detector	sensor.other.lane_invasion	carla.LaneInvasionEvent	记录换道事件
Obstacle detector	sensor.other.obstacle	carla.ObstacleDetectionEvent per obstacle	每当前方有障碍物时，就会注册一个事件



常见传感器配置-传感器简介

Lidar传感器的输出数据类型也与camera完全不同。Lidar大类传感器下有两种传感器，分别是ray_cast和ray_cast_semantic

传感器	蓝图	输出	功能
LIDAR sensor	sensor.lidar.ray_cast	carla.LidarMeasurement per step	模拟使用光线投射实现的旋转激光雷达。这些点是通过为垂直 FOV 中分布的每个通道添加激光来计算的。通过计算激光雷达在一帧中旋转的水平角度来模拟旋转。 每通道每步点数 = 每秒点数 / (FPS * 通道数)
Semantic LIDAR sensor	sensor.lidar.ray_cast_semantic	carla.SemanticLidarMeasurement per step	相比激光雷达，还额外提供语义信息



常见传感器配置-传感器简介

other类的传感器提供了其他各种各样子不同的功能，例如导航、物理特性测量。other类的传感器和camera类类似，也是在每次模拟环境更新时获取信息。

传感器	蓝图	输出	功能
Radar sensor	sensor.other.radar	carla.RadarMeasurement per step	传感器会创建一个圆锥视图，该视图会转化为一个二维点图，显示视线内的元素及其相对于传感器的速度。这可用于塑造元素形状并评估其运动 and 方向。
GNSS sensor	sensor.other.gnss	carla.GNSSMeasurement per step	获取当前所在位置（经纬度、高度、时间等），计算方法是将度量位置添加到 OpenDRIVE 地图定义中定义的初始地理参考位置。
IMU sensor	sensor.other.imu	carla.IMUMeasurement per step	提供加速度计、陀螺仪和指南针将为父对象检索的测量值。数据是从对象的当前状态收集的。



常见传感器配置-绑定传感器

Carla提供了三种不同的 AttachmentType:

- Rigid: 这种绑定方式是紧密固定被绑定物体与绑定物体的方法，是完全固定的绑定方式，如果将相机绑定到车辆上，那么车辆颠簸等都会被传递到相机上，适合想要在仿真中获得准确数据的情况。
- SpringArm: 这种绑定方式会根据不同情况伸长或者缩短与被绑定物体的距离，就像弹簧一样，适合想要获得没有抖动的平滑录像时使用。
- SpringArmGhost: 这种绑定方式和第二种方式很相似，但是这种方式不会进行碰撞检测，意味着通过这个方式绑定的actor可以穿过墙体或者其他模型，同样也是适合想要获得没有抖动的平滑录像时使用。0.9.14版本中出现

```
camera = world.spawn_actor(camera_bp, camera_transform,  
                             attach_to=ego_vehicle,  
                             attachment_type=carla.AttachmentType.SpringArm)
```



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

RGB Camera

Carla提供了非常多的可以设置的属性供我们自定义我们想要的相机效果，我们可以通过调整各种属性来模拟现实中相机在各个不同环境条件下的反应，例如视角，快门速度等。官方文档链接在这里

https://link.zhihu.com/?target=https%3A//carla.readthedocs.io/en/latest/ref_sensors/%23rgb-camera

Carla中RGB相机有着如下的基本设定，一般情况下我们对于基本设定进行调整就能够满足我们对于传感器的自定义要求，注意进行属性设定需要对于生成所用蓝图进行设定，设定属性需要在生成之前，生成后就无法更改属性了。

蓝图属性	数据类型	默认值	描述
bloom_intensity	float	0.675	镜头炫光强度，设置为0.0可以关闭
fov	float	90.0	水平视角，单位：度
fstop	float	1.4	光圈大小，fstop值越小光圈越大，景深越浅，一般情况下最大光圈（即最小fstop值）为1.2
image_size_x	int	800	图像宽度，单位：像素
image_size_y	int	600	图像高度，单位：像素
iso	float	100.0	感光度
gamma	float	2.2	相机目标gamma值
lens_flare_intensity	float	0.1	镜头鬼影强度，设置为0.0可以关闭
sensor_tick	float	0.0	图像采集间隔，单位：tick
shutter_speed	float	200.0	快门速度，单位 1/shutter_speed 秒

驾驶之心



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

更多高级属性见文档，核心代码如下：

```
camera_bp = blueprint_library.find('sensor.camera.rgb')#找到相机的蓝图
camera_transform = carla.Transform(carla.Location(x=1.5, z=2.4))#设置相对位置，第一种，可以看正前方，单位:m
camera_transform = carla.Transform(carla.Location(x=-8, y=0, z=3),#第二种，可以看全局
                                   carla.Rotation(pitch=10, yaw=0, roll=0))
camera = world.spawn_actor(camera_bp, camera_transform, attach_to=ego_vehicle,
                           attachment_type=carla.AttachmentType.SpringArmGhost)#安装到车上
#camera.listen(lambda image:
image.save_to_disk(os.path.join('/home/wj/PycharmProjects/carla/output_pic', '%06d.png' %
image.frame)))#保存图片
camera.listen(lambda image: process_img(image))#也可以通过pygame显示图片
camera.destroy()#最后记得销毁
```

完整代码分析

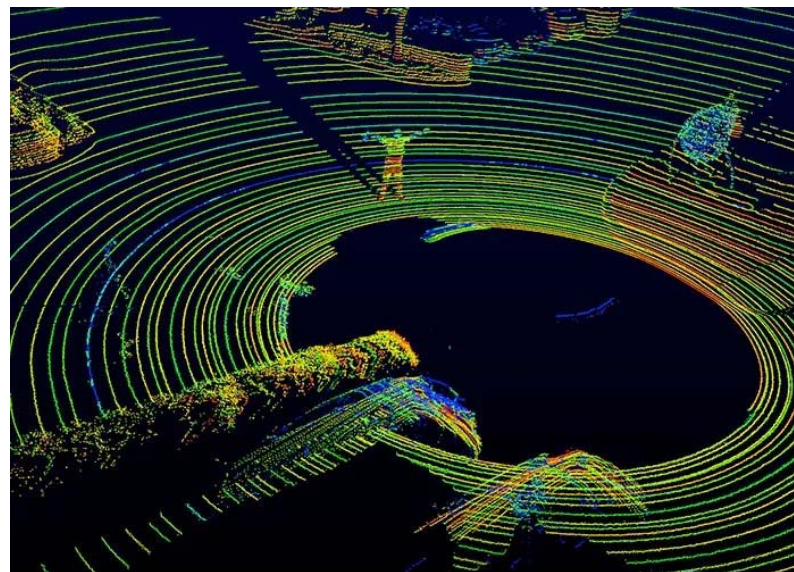
自动驾驶之心



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

机械激光雷达 (LiDAR) 是一种测距技术，它利用激光脉冲向目标物体发射并接收反射回来的激光，通过计算激光往返时间来测量物体的距离。机械激光雷达通过旋转发射和接收激光，覆盖周围环境，生成高精度的三维点云地图。机械激光雷达的工作原理和主要参数：激光雷达 (LiDAR) 的性能和功能可以通过多种参数来描述，这些参数决定了激光雷达的探测能力、分辨率和适用范围。下面是一些关键的激光雷达参数及其说明：

1. 射线数量 (Channels)：指的是激光雷达可以同时发射和接收的激光束数量。射线数量越多，设备能够在单次扫描中捕获的细节就越多，从而提高了点云的密度和测量的精度。
2. 探测距离 (Range)：激光雷达能够检测目标的最大距离，这通常取决于激光的功率、接收器的灵敏度和目标物体的反射率。探测距离可以从几米到几千米不等，具体取决于激光雷达的设计和应用场景。
3. 点云密度 (Point Density)：这是指激光雷达在单位面积内生成的点的数量。点云密度越高，生成的3D模型就越精细。点云密度受到多个因素的影响，包括射线数量、扫描角度范围、激光雷达的工作距离以及扫描速度。
4. 激光雷达转动速率 (Rotation Speed)：对于机械式激光雷达，转动速率指的是激光雷达单元转动的速度，通常以转/分钟 (rpm) 来衡量。转速越高，单位时间内扫描的区域就越大，但可能会影响到点云的密度和测量精度。
5. 角度分辨率 (Angular Resolution)：这是指激光雷达能够区分两个相邻点的最小角度差异，通常以度或弧度来衡量。角度分辨率越高，激光雷达捕获的细节就越多，但相应地会减少单次扫描覆盖的总范围。
6. 每秒点数 (Points Per Second)：这个参数指的是激光雷达每秒可以生成的点的数量，它直接影响到点云的密度和激光雷达的测量速率。每秒点数越高，激光雷达生成的点云就越密集，能够更快地完成扫描。



Lidar sensor通过UE4引擎中的ray-casting模拟了一个旋转的激光雷达，传感器会根据设定的通道数(channel)为每个通道添加一条射线并且计算每条射线的旋转角度来模拟出旋转的激光雷达，通过每一次步都对每个射线进行ray-cast计算，最后传感器会得到一份点云图。Lidar传感器会每步输出一个 [carla.LidarMeasurement] 数据，这个数据中包含一个存储了在1/FPS秒钟所有生成的点的信息的包，我们也可以将这个包含点的信息的包以list形式输出出来：

```
for location in lidar_measurement:  
    print(location)
```



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

Lidar传感器会输出一个4D点云图, XYZI, 的前3个是x, y, z坐标, 而第四个属性是激光密度, 强度(intensity)为了模拟更加真实的激光雷达传感器, 点云中的点会丢失, 减少(drop off)以模拟现实中的外部扰动, 点的drop off总共由两部分组成, 一部分是基础 General drop off, 另一种是基于intensity的 Instensity-based drop-off。 General drop off可以通过设定 dropoff_general_rate`来更改drop off 比例, Instensity-based drop-off可以通过更改上下限: dropoff_zero_intensity和 dropoff_intensity_limit来更改drop off 的比例。除此之外, 激光雷达还可以向输出结果中加入噪声来模拟真实的场景以使模拟仿真更加接近现实。下面我们看看激光雷达所提供的**各种属性**

蓝图属性	数据类型	默认值	描述
channels	int	32	激光射线数量
range	float	10.0	激光雷达最大探测距离, 单位: 米 (CARLA 0.9.6或更早版本单位为厘米).
points_per_second	int	56000	每秒所有激光所产生的点的数量
rotation_frequency	float	10.0	激光雷达转动速率
upper_fov	float	10.0	最上方激光的角度, 单位: 度
lower_fov	float	-30.0	最下方激光的角度, 单位: 度
horizontal_fov	float	360.0	水平范围激光雷达检测范围, 单位: 度
atmosphere_attenuation_rate	float	0.004	激光雷达衰减常数, 控制每米的intensity loss
dropoff_general_rate	float	0.45	基础drop off率
dropoff_intensity_limit	float	0.8	Instensity-based drop-off 上限
dropoff_zero_intensity	float	0.4	Instensity-based drop-off 下限
sensor_tick	float	0.0	传感器更新速率, 单位: ticks
noise_stddev	float	0.0	噪声添加标准差



常见传感器配置-部署常用传感器（RGB Camera、Lidar、GNSS、IMU）

激光雷达输出属性

传感器数据属性	数据类型	描述
frame	int	传感器获取数据时的帧编号（第几帧时捕捉的画面）
timestamp	double	从开始模拟到数据捕捉的模拟时间
transform	carla.Transform	传感器在模拟环境下的位置以及角度
horizontal_angle	float	当前帧激光雷达所在平面角度
channels	int	通道数，激光数
get_point_count(channel)	int	当前帧输入通道中点的数量
raw_data	bytes	32bits 浮点数，存储每个点的X，Y，Z值等

激光雷达传感器所输出的数据是.ply文件，可以直接导入到**meshlab**中进行查看。在Ubuntu上进行meshlab的安装也很简单：

```
sudo apt-get install meshlab
meshlab XXX.ply
```



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

核心代码如下:

```
lidar_bp = blueprint_library.find('sensor.lidar.ray_cast') # 查找Lidar传感器的蓝图
lidar_bp.set_attribute('range', '100') # 设置Lidar的检测范围为100米
lidar_bp.set_attribute('rotation_frequency', '10') # 设置Lidar的旋转频率为10Hz
lidar_bp.set_attribute('channels', '32') # 设置Lidar的通道数为32
lidar_bp.set_attribute('points_per_second', '56000') # 设置每秒的点数
# 设置Lidar传感器相对于车辆的位置
lidar_transform = carla.Transform(carla.Location(x=0, z=2.5)) # Lidar相对于车辆的位置

# 在车上安装Lidar传感器
lidar = world.spawn_actor(lidar_bp, lidar_transform, attach_to=ego_vehicle)
# Lidar数据监听
lidar.listen(lambda data: process_lidar(data))

lidar.stop() # 停止Lidar数据监听
lidar.destroy() # 销毁Lidar传感器
```

完整代码一: 保存每一帧的ply文件并查看

完整代码二: 在pygame中实时现实激光雷达点云 (二维), 三维显示在后续课程会讲到

自动驾驶之心



常见传感器配置-部署常用传感器（RGB Camera、Lidar、GNSS、IMU）

全球导航卫星系统（GNSS）是一种提供全球定位、导航和时间测量服务的卫星导航系统。它利用地球轨道上的卫星群向地面用户发送信号，用户设备通过接收来自不同卫星的信号来计算其位置、速度和时间。

在Carla中GNSS传感器是一个关键的组件，用于模拟现实世界中车辆使用GNSS进行定位和导航的能力，提供经纬度、高度和时间信息，以下是GNSS传感器的属性和数据的属性：

蓝图属性	数据类型	默认	描述
noise_alt_bias	float	0.0	海拔高度噪声模型中的平均参数
noise_alt_stddev	float	0.0	海拔高度噪声模型中的标准偏差参数。
noise_lat_bias	float	0.0	纬度噪声模型中的平均参数。
noise_lat_stddev	float	0.0	纬度噪声模型中的标准偏差参数。
noise_lon_bias	float	0.0	经度噪声模型中的平均参数。
noise_lon_stddev	float	0.0	经度噪声模型中的标准偏差参数。
noise_seed	int	0	伪随机数生成器的初始化程序。
sensor_tick	float	0.0	传感器捕获之间的模拟秒数。

传感器数据属性	数据类型	描述
frame	int	进行测量时的帧编号。
timestamp	double	仿真时间步。
transform	carla.Transform	测量时传感器在世界坐标中的位置和旋转。
latitude	double	纬度
longitude	double	经度
altitude	double	高度



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、**GNSS**、IMU)

部署GNSS的核心代码

```
gnss_bp = world.get_blueprint_library().find('sensor.other.gnss')
gnss_bp.set_attribute('sensor_tick', '0.1')
gnss_sensor = world.spawn_actor(gnss_bp, carla.Transform(), attach_to=vehicle)
gnss_sensor.listen(lambda data: logging.info('GNSS Latitude: %f, Longitude: %f,
Altitude: %f' % (data.latitude, data.longitude, data.altitude)))
```

完整代码分析，给车辆以10HZ的频率部署GNSS，并且以log的形式展示当前的经纬度高度



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

在CARLA仿真环境中，惯性测量单元 (IMU) 传感器模拟了现实世界中的IMU设备，提供了关于车辆运动状态的重要信息。IMU是自动驾驶系统中不可或缺的组成部分，因为它能够提供车辆的加速度、角速度和方向等数据。以下是CARLA中IMU传感器的具体作用：

1. 提供加速度信息

- IMU传感器测量并提供车辆三个空间维度（通常是X、Y、Z轴）上的加速度信息。这对于理解车辆的动态行为（如加速、减速和转弯）至关重要。

2. 测量角速度

- IMU还能测量车辆绕其轴线旋转的速率，即角速度。这对于评估车辆的稳定性和制定控制策略非常有用。

3. 提供方向信息

- 通过结合加速度和角速度的数据，IMU可以帮助确定车辆的方向，即车辆相对于地球坐标系的朝向。这对于导航和保持车辆在预定路径上非常重要。

4. 增强车辆感知能力

- 尽管自动驾驶车辆使用多种传感器（如摄像头、激光雷达和雷达）来感知周围环境，但IMU提供的内部运动状态数据对于完善车辆的感知系统和提高感知数据的准确性非常重要。

5. 支持多传感器数据融合

- IMU数据常与其他传感器数据结合使用，进行多传感器数据融合。这种融合技术可以提高车辆的定位精度、运动估计和环境感知能力，特别是在GPS信号不稳定或缺失的情况下。

6. 测试和验证自动驾驶算法

- 在CARLA仿真环境中使用IMU传感器，可以在没有实际车辆或在受控环境之外的情况下，测试和验证自动驾驶算法。这有助于在安全的虚拟环境中识别和解决潜在的问题，从而降低了开发成本和风险。

自动驾驶之心



常见传感器配置-部署常用传感器 (RGB Camera、Lidar、GNSS、IMU)

蓝图属性与数据属性

完整代码分析，给车辆以10HZ的频率部署IMU，并且以log的形式展示当前的加速度，陀螺仪偏差和朝向

蓝图属性	类型	默认值	描述
noise_accel_stddev_x	float	0.0	加速度（X 轴）噪声模型中的标准偏差参数。
noise_accel_stddev_y	float	0.0	加速度（Y 轴）噪声模型中的标准偏差参数。
noise_accel_stddev_z	float	0.0	加速度（Z 轴）噪声模型中的标准偏差参数。
noise_gyro_bias_x	float	0.0	陀螺仪噪声模型中的平均参数（X 轴）。
noise_gyro_bias_y	float	0.0	陀螺仪噪声模型中的平均参数（Y 轴）。
noise_gyro_bias_z	float	0.0	陀螺仪噪声模型中的平均参数（Z 轴）。
noise_gyro_stddev_x	float	0.0	陀螺仪噪声模型中的标准偏差参数（X 轴）。
noise_gyro_stddev_y	float	0.0	陀螺仪噪声模型中的标准偏差参数（Y 轴）。
noise_gyro_stddev_z	float	0.0	陀螺仪噪声模型中的标准偏差参数（Z 轴）。
noise_seed	int	0	伪随机数生成器的初始化程序。
sensor_tick	float	0.0	传感器捕获之间的模拟秒数

传感器数据属性	类型	描述
frame	int	进行测量时的帧编号。
timestamp	double	仿真时间步
transform	carla.Transform	测量时传感器在世界坐标中的位置和旋转。
accelerometer	carla.Vector3D	测量线性加速度（以 m/s^2 为单位）。
gyroscope	carla.Vector3D	测量角速度（以 rad/sec 为单位）
compass	float	以弧度为单位的方向。UE 中的北为 (0.0, -1.0, 0.0)。





37

Carla上手实战

从0到1实现自己的炫酷小车

在这里总结一下之前学过的加载地图、同步设置、交通管理器设置、生成车辆、配置传感器(这里只配置了相机)，此外增加一个利用pygame来控制键盘从而控制我们车辆行走的程序，新知识点是关于**pygame的使用**，包括车辆的控制和展示相机拍到的画面。
首先还是完成基础环境，**加载地图，同步设置，交通管理器设置，生成多辆自动驾驶汽车，以及给ego车辆安装相机**等操作

```
def main():
    client = carla.Client('localhost', 2000)#用户通过Client载体与python API与仿真环境交互
    # 们第一部就是要创建Client

    client.load_world('Town03')#选择加载的地图，0 3 是默认的
    client.set_timeout(5.0)#设置一个timeout时间防止连接时间过久
    world = client.get_world()#构建的Client来获取仿真世界 (world)

    # 同步模式设置
    settings = world.get_settings()
    settings.synchronous_mode = True# 启用同步模式
    settings.fixed_delta_seconds = 0.05# 设置每一帧的时间，这里为0.05秒
    world.apply_settings(settings)# 应用设置

    # 配置并启用traffic manager的同步模式
    traffic_manager = client.get_trafficmanager(8000)
    traffic_manager.set_synchronous_mode(True)

    # 设置种子，以便必要时行为可以重复
    traffic_manager.set_random_device_seed(0)
    random.seed(0)

    # 获取蓝图库并设置ego_vehicle蓝图
    ego_vehicle_bp = world.get_blueprint_library().find('vehicle.mercedes-benz.coupe')#找到奔驰的蓝图

    ego_vehicle_bp.set_attribute('color', '0, 0, 200')#给车加上特定的颜色
    ego_vehicle_bp.set_attribute('role_name', 'hero')# 设置主车蓝图的属性中的角色名

    # 获取生成点
    spawn_points = world.get_map().get_spawn_points()
    ego_spawn_point = random.choice(spawn_points) # 随机选择一个初始生成点

    # 选择生成点并生成车辆
    ego_vehicle = world.spawn_actor(ego_vehicle_bp, ego_spawn_point)#在这个位置生成汽车
    ego_vehicle.set_autopilot(True, 8000)#autopilot

    # 获取车辆蓝图
    blueprint_library = world.get_blueprint_library()
    vehicle_blueprints = blueprint_library.filter('vehicle.audi.*')

    spawn_points.remove(ego_spawn_point) # 从可用生成点列表中移除已用点
```

从0到1实现自己的炫酷小车

```
# 创建车辆批量操作列表
number_of_vehicles = 50
batch = []
for i in range(min(number_of_vehicles, len(spawn_points))):
    spawn_point = random.choice(spawn_points) # 随机选择生成点
    vehicle_bp = random.choice(vehicle_blueprints) # 随机选择一辆车的蓝图
    batch.append(carla.command.SpawnActor(vehicle_bp, spawn_point))
    spawn_points.remove(spawn_point)

# 执行批量操作
other_vehicle_list = []
vehicles_id_list = []
for i, response in enumerate(client.apply_batch_sync(batch, True)):
    if response.error:
        logging.error(response.error)
    else:
        vehicles_id_list.append(response.actor_id)
        # 设置车辆自动驾驶
        vehicle = world.get_actor(response.actor_id)
        other_vehicle_list.append(vehicle)
        vehicle.set_autopilot(True, traffic_manager.get_port())
```



从0到1实现自己的炫酷小车

```
# 设置观察者视角
spectator = world.get_spectator()#建立spectator对象

# 相机设置
camera_bp = blueprint_library.find('sensor.camera.rgb')#找到相机的蓝图
camera_transform = carla.Transform(carla.Location(x=-5, y=0, z=3),#第二种,可以看全局
                                   carla.Rotation(pitch=-20, yaw=0, roll=0))
camera = world.spawn_actor(camera_bp, camera_transform,attach_to=ego_vehicle)#安装到车上
```

接着是利用pygame对键盘进行监听,上下左右控制油门转角, 回车切换自动驾驶或手动, 完成一个监听键盘的函数, 根据键盘的输入, 转化为控制指令

```
def parse_control(self, event):
    if event.type == pygame.KEYDOWN:
        # 当按下回车键时切换自动驾驶状态
        if event.key == pygame.K_RETURN:
            self._autopilot = not self._autopilot
            self._vehicle.set_autopilot(self._autopilot)
        # 按下方向键上键时将油门属性设定为True
        if event.key == pygame.K_UP:
            self._throttle = True
        # 按下方向键下键时将刹车属性设定为True
        if event.key == pygame.K_DOWN:
            self._brake = True
        # 按下方向键右键时将方向盘属性设定为1
        if event.key == pygame.K_RIGHT:
            self._steer = 1
        # 按下方向键左键时将方向盘属性设定为-1
        if event.key == pygame.K_LEFT:
            self._steer = -1
```

```
# 抬起按键时将属性重设为默认值
if event.type == pygame.KEYUP:
    if event.key == pygame.K_UP:
        self._throttle = False
    if event.key == pygame.K_DOWN:
        self._brake = False
    if event.key == pygame.K_RIGHT:
        self._steer = None
    if event.key == pygame.K_LEFT:
        self._steer = None
```

自动驾驶之心



从0到1实现自己的炫酷小车

有了控制指令后，我们还需要给出具体的控制量，油门刹车转角具体给多少，用于控制车辆的运动

```
def process_control(self):  
    if self._throttle:  
        self._control.throttle = min(self._control.throttle + 0.05, 1)  
        self._control.gear = 1  
        self._control.brake = False  
    elif not self._brake:  
        self._control.throttle = 0.0
```

发出油门指令时，油门逐步加大，但不能超过1，没有油门且没有刹车是，油门给0

```
    if self._brake:  
        velocity = self._vehicle.get_velocity()  
        if math.sqrt(velocity.x**2 + velocity.y**2 + velocity.z**2) < 0.01 and not  
self._control.reverse:# 当按住方向下键并且车辆处于静止状态，切换到倒车档位并加速  
            self._control.brake = 0.0  
            self._control.gear = 1  
            self._control.reverse = True  
            self._control.throttle = min(self._control.throttle + 0.1, 1)  
        elif self._control.reverse:  
            self._control.throttle = min(self._control.throttle + 0.1, 1)  
        else:  
            self._control.throttle = 0.0  
            self._control.brake = min(self._control.brake + 0.3, 1)  
    else:  
        self._control.brake = 0.0
```

发出刹车指令时，判断车辆是否处于静止状态，若是，切换倒车档加速（进一步需要判断倒车档）；若否，先将油门量为0，再逐渐加大刹车量

自动驾驶之心



从0到1实现自己的炫酷小车

有了控制指令后，我们还需要给出具体的控制量，油门刹车转角具体给多少，用于控制车辆的运动

```
if self._steer is not None:
    if self._steer == 1:
        self._steer_cache += 0.03
    if self._steer == -1:
        self._steer_cache -= 0.03
    min(0.7, max(-0.7, self._steer_cache))
    self._control.steer = round(self._steer_cache, 1)
else:
    if self._steer_cache > 0.0:
        self._steer_cache *= 0.2
    if self._steer_cache < 0.0:
        self._steer_cache *= 0.2
    if 0.01 > self._steer_cache > -0.01:
        self._steer_cache = 0.0
    self._control.steer = round(self._steer_cache, 1)

# 将存储在self._control的控制信息应用
self._vehicle.apply_control(self._control)
```

发出转角指令时，根据方向进行对应的加减，没有转角指令时进行修正。



从0到1实现自己的炫酷小车

给车辆绑定相机，并且可以通过TAB按键来切换ego_vehicle

```
# 监听按键信息
for event in pygame.event.get():
    # 窗口关闭则退出循环
    if event.type == pygame.QUIT:
        crashed = True
    # 将监听到的按键信息转化成控制信息
    controlObject.parse_control(event)
    if event.type == pygame.KEYUP:
        # 按下tab键切换到其他车辆
        if event.key == pygame.K_TAB:
            ego_vehicle.set_autopilot(True)
            ego_vehicle = random.choice(other_vehicle_list)
            # 确保车辆仍存活
            if ego_vehicle.is_alive:
                # 清除原本摄像头
                camera.stop()
                camera.destroy()
                # 生成新摄像头并绑定到新主车上
                controlObject = ControlObject(ego_vehicle)
                camera = world.spawn_actor(camera_bp, camera_transform,
attach_to=ego_vehicle)
                camera.listen(lambda image: pygame_callback(image,
renderObject))
```

当按了TAB键时，先将自车变为自动驾驶模式，然后重新选取一辆车，如果该车存在，杀死原来的相机，在这辆车上安装新的相机

自动驾驶之心



从0到1实现自己的炫酷小车

Pygame显示相机图像相关的代码

```
def pygame_callback(data,obj):# 相机传感器回调,将相机的原始数据重塑为2D RGB,并应用于PyGame表面
    img = np.reshape(np.copy(data.raw_data), (data.height, data.width, 4))
    img = img[:, :, :3]
    img = img[:, :, ::-1]
    obj.surface = pygame.surfarray.make_surface(img.swapaxes(0,1))
```

```
# pygame初始化用于显示
pygame.init()
gameDisplay = pygame.display.set_mode((image_w,image_h), pygame.HWSURFACE |
pygame.DOUBLEBUF)
gameDisplay.fill((0,0,0))
gameDisplay.blit(renderObject.surface, (0,0))
pygame.display.flip()
```

这个函数接收来自相机传感器的数据data和一个渲染对象obj作为参数。首先，它将data.raw_data（相机传感器的原始数据）重塑为一个具有正确形状的NumPy数组，该数组具有形状(height, width, 4)，其中4代表RGBA通道。接着，它去除了Alpha通道（即：img[:, :, :3]），因为Pygame仅需要RGB值。然后，将图像的颜色通道从BGR转换为RGB，因为CARLA通常以BGR格式提供图像数据，而Pygame则需要RGB格式。最后，使用pygame.surfarray.make_surface将处理后的图像转换为Pygame的Surface对象，并交换了图像的x和y轴，以符合Pygame的显示方式

首先，通过pygame.init()初始化Pygame。然后，创建一个Pygame窗口（或称之为“显示”），其尺寸由相机的图像尺寸定义（image_w和image_h）。使用gameDisplay.fill((0,0,0))将窗口背景填充为黑色。使用gameDisplay.blit(renderObject.surface, (0,0))将图像绘制到窗口上，其中renderObject.surface是由pygame_callback函数创建的Pygame Surface对象。最后，通过pygame.display.flip()更新整个窗口的显示，以显示最新的图像帧。

自动驾驶之心



从0到1实现自己的炫酷小车

程序运行完的清理

```
finally:
    # 退出前将仿真设置还原为非同步模式
    settings.synchronous_mode = False
    settings.fixed_delta_seconds = None
    world.apply_settings(settings)

    print("清理other车辆...")
    for vehicle in world.get_actors().filter('*vehicle*'):
        vehicle.destroy()
    print("other车辆清理完成")

    print("关闭Pygame窗口...")
    pygame.quit()

    print("清理camera...")
    camera.destroy()
    print("camera清理完成")
```

退出同步模式
清理车辆
关闭Pygame
清理传感器

完整代码运行

自动驾驶之心



地图切换与交通参与者的设计-自带地图的描述

TOWN	描述
town01	具有所有“T型交叉路口”的基本城镇布局。
town02	与Town01类似，但更小一些
town03	最复杂的城镇，有一个5车道的交叉路口，一个环形路口，不平坦的路面，一个隧道等等。
town04	高速公路与小镇的循环
town05	方形网格城镇，有十字路口和一座桥。每个方向都有多条车道。用于测试车辆变道算法。
town06	有许多出入口的高速公路。
town07	乡村环境，道路狭窄，几乎没有红绿灯
town10	一个带有不同环境的城市环境，例如林荫大道，纹理更加逼真。



地图切换与交通参与者的设计-自带地图的导入

```
cd ~/carla/Import  
./ImportAssets.sh
```



地图切换与交通参与者的设计-OpenDrive格式地图的导入

```
#coding=utf-8
import carla
import io
def main():
    # 连接到CARLA服务器
    client = carla.Client('localhost', 2000)
    client.set_timeout(10.0)

    # 加载自定义地图
    xodr_path = str('/home/wj/carla_0.9.10.1/PythonAPI/util/opendrive/TownBig.xodr')
    with io.open(xodr_path, 'r', encoding='utf-8') as od_file:
        data = od_file.read()
        data = data.encode('utf-8')
        vertex_distance = 1.0 # in meters
        max_road_length = 500.0 # in meters
        wall_height = 0.5 # in meters
        extra_width = 1.0 # in meters
        world = client.generate_opendrive_world(
            data, carla.OpendriveGenerationParameters(
                vertex_distance=vertex_distance,
                max_road_length=max_road_length,
                wall_height=wall_height,
                additional_width=extra_width,
                smooth_junctions=True,
                enable_mesh_visibility=True))
    print("the current world is:", world)
```

OpenDRIVE是一种用于道路网络描述的开放文件格式，专为高精度地图设计，广泛用于驾驶模拟器、自动驾驶系统的测试和验证中。这种格式能够详细描述道路的几何、拓扑结构、标志、信号灯、车道信息等，使其成为自动驾驶研究和开发的重要工具。

定义一系列参数，用于控制OpenDRIVE地图在CARLA中的生成方式。这些参数包括顶点距离、最大道路长度、墙高和额外宽度等。

调用client.generate_opendrive_world方法，传入OpenDRIVE数据和上述参数，生成相应的CARLA世界。OpendriveGenerationParameters对象中的参数用于细化道路网的生成，例如通过平滑交叉口、调整道路宽度等。

自动驾驶之心



38

Carla知识点梳理

算法的架构-客户端与服务端

Carla主要分为**Server**与**Client**两个模块，Server端用来建立这个仿真世界，而Client端则是由用户控制，用来调整、变化这个仿真世界。

Server: Server端负责任何与仿真本身相关的事情：从3D渲染汽车、街道、建筑，传感器模型的构建，到物理计算等等。它就像一个造物主，将整个世界建造出来，并且根据Client 的外来指令更新这个世界。它本身是基于[UnrealEnigne]做出的3D渲染。

Client: 如果Server构造了整个世界，那么这个世界不同时刻到底该如何运转（比如天气是什么样，有多少辆车在跑，速度是多少）则是由Client端控制的。用户通过书写Python脚本（C++ 也可以）来向Server端输送指令指导世界的变化，Server根据用户的指令去执行。（可以理解为Client端耍耍嘴皮子下个指令，咱们的造物主亲力亲为去执行这些指令。）另外，Client端也可以接受Server端的信息，譬如某个照相机拍到的路面图片。



算法的架构-核心模块作用

Traffic Manager: 模拟类似现实世界负责的交通环境。通过这个模块，用户可以定义N多不同车型、不同行为模式、不同速度的车辆在路上与Ego-Vehicle交互。

Sensors: Carla里面有各种各样模拟真实世界的传感器模型，包括相机、激光雷达、声波雷达、IMU、GNSS等等。为了让仿真更接近真实世界，它里面的相机拍出的照片甚至还有畸变和动态模糊效果。用户一般将这些Sensor attach到不同的车辆上来收集各种数据。

Recorder: 用来记录仿真每一个时刻 (Step)的状态，可以用来回顾、复现等等。

ROS bridge: 这个模块可以让Carla与ROS还有Autoware交互，正是这个模块的存在使得在仿真里测试你的自动驾驶系统变得可能。

Open Asset Import Library: 用于导入多种3D模型格式到CARLA的仿真环境中。Assimp支持包括但不限于FBX, COLLADA, 3DS, OBJ等多种格式。通过使用这个模块，用户可以将自定义的3D模型（如建筑物、道路标志、其他静态物体等）导入到CARLA中，从而丰富仿真环境的细节和真实感。



automatic_control中的算法解析

这个脚本是一个CARLA中的全功能自动控制仿真。它设置环境、车辆、控制和传感器，然后运行仿真，允许通过键盘控制进行交互，并在屏幕上显示仿真数据。用户可以通过不同的参数自定义脚本，并将其作为在CARLA模拟器中进一步实验自动驾驶算法的基础。



automatic_control中的算法解析-BehaviorAgent源码分析

前面介绍了通过蓝图生成车辆，在此基础上，进一步介绍`BehaviorAgent`类，相当于车辆的**大脑**，决定车辆的行为方式。在CARLA中，`BehaviorAgent`类通常提供不同的驾驶行为或风格，包括"cautious"（谨慎）、"normal"（正常）或"aggressive"（侵略）等。这些行为风格会影响代理在模拟环境中的驾驶方式，例如限速、与前车保持的安全时间、遵守交通规则等。当你初始化一个`BehaviorAgent`时，你可以根据需要的驾驶风格传递一个字符串参数。根据这个参数，`BehaviorAgent`将调整其内部设置以模拟不同的行为。以下是如何根据不同的驾驶风格字符串初始化`BehaviorAgent`的简单例子：

```
from agents.navigation.behavior_agent import BehaviorAgent

# 初始化CARLA世界和车辆...

# 根据用户输入选择不同的驾驶风格
behavior_style = input("请输入驾驶风格 (cautious, normal, aggressive): ")

# 创建BehaviorAgent
agent = BehaviorAgent(vehicle, behavior=behavior_style)

# 其余的仿真代码...
```



automatic_control中的算法解析-全局路径规划代码解析

在automatic_control中，有关全局规划的代码只有一行

```
agent.set_destination(agent.vehicle.get_location(), destination, clean=True)
```

这行代码的作用是告诉`BehaviorAgent`代理：从当前控制的车辆所在位置开始，规划一条到给定`destination`的路径，并确保在开始之前清除任何旧的路径或目标信息。这是自动驾驶仿真中常见的步骤，用于初始化或更新车辆的行进目标。这个函数位于
~/carla_0.9.10.1/PythonAPI/carla/agents/navigation/behavior_agent.py中

```
def set_destination(self, start_location, end_location, clean=False):  
    ...  
    self.start_waypoint = self._map.get_waypoint(start_location)  
    self.end_waypoint = self._map.get_waypoint(end_location)  
  
    route_trace = self._trace_route(self.start_waypoint, self.end_waypoint)  
  
    self._local_planner.set_global_plan(route_trace, clean)
```

在这段代码中首先，代码将`start\_location`和`end\_location`转换成CARLA的路点（`Waypoint`）。这些路点是地图上的具体位置，全局路线规划器将用它们来计算路径。



automatic_control中的算法解析-全局路径规划代码解析

使用`\_trace\_route`方法计算路径：`\_trace\_route`方法接受起始和结束路点作为输入，并返回一系列的路点，代表从起始位置到目标位置的最优路径。它内部调用了全局路线规划器（`GlobalRoutePlanner`）的`trace\_route`方法来计算这个路径。

```
route_trace = self._trace_route(self.start_waypoint, self.end_waypoint)
```

设置局部规划器的全局计划：最后，计算出的路径（路点序列）被传递给局部规划器（`LocalPlanner`），它将使用这些路点来导航和执行驾驶决策。

```
self._local_planner.set_global_plan(route_trace, clean)
```

`\_trace\_route`方法是`set\_destination`中用来与全局路线规划器交互的关键部分。它确保了`BehaviorAgent`可以根据CARLA的道路网络来规划路径。下面是对这个方法的详细分析：

```
def _trace_route(self, start_waypoint, end_waypoint):
    if self._grp is None:
        wld = self.vehicle.get_world()
        dao = GlobalRoutePlannerDAO(wld.get_map(),
                                     sampling_resolution=self._sampling_resolution)
        grp = GlobalRoutePlanner(dao)
        grp.setup()
        self._grp = grp

    route = self._grp.trace_route(
        start_waypoint.transform.location,
        end_waypoint.transform.location)

    return route
```

自动驾驶之心



automatic_control中的算法解析-全局路径规划代码解析

GlobalRoutePlannerDAO负责访问地图，输入为地图和采样精度，输出为拓扑数据、道路节点和道路属性。通过预处理和封装地图数据，减少了直接处理数据的数据量和复杂度。

```
dao = GlobalRoutePlannerDAO(wld.get_map(),
sampling_resolution=self._sampling_resolution)
```

初始化GlobalRoutePlanner，使用地图数据和采样分辨率来设置道路网络，setup()函数如下：

```
grp = GlobalRoutePlanner(dao)
grp.setup()
```

```
def setup(self):
    """
    Performs initial server data lookup for detailed topology
    and builds graph representation of the world map.
    """
    self._topology = self._dao.get_topology()
    self._graph, self._id_map, self._road_id_to_edge = self._build_graph()
    self._find_loose_ends()
    self._lane_change_link()
```

self._topology = self._dao.get_topology()用于提取地图拓扑结构，具体格式为：

```
[{entry:Waypoint1, exit:Waypoint2, path:[Waypoint1_1, Waypoint1_2,.....]},
 {entry:Waypoint1, exit:Waypoint2, path:[Waypoint1_1, Waypoint1_2,.....]},
 .....
```

- **entry** (Waypoint1): 代表道路的入口点，是这段道路的开始。它是一个`Waypoint`对象，包含了该点的详细信息，如坐标、车道信息等。
- **exit** (Waypoint2): 代表道路的出口点，是这段道路的结束。它也是一个`Waypoint`对象，包含了该点的详细信息。
- **path** ([Waypoint1_1, Waypoint1_2,]): 表示从入口到出口的路径点list，间距是由前面的sampling resolution决定的。每个元素都是一个`Waypoint`对象，表示路径上的一个具体位置（lane中央）。这些路径点一起形成了从入口到出口的完整道路。



automatic_control中的算法解析-全局路径规划代码解析

这行代码是在构建一个图结构，用来表示路网的拓扑结构。这个图结构是进行路径规划和导航时的基础。`\_build\_graph()`是一个方法，其目的是构建代表整个路网的图（Graph）。在这个图中，节点可能代表路口或道路段的特定点，边代表道路连接这些点。返回的图通常是一个有向图，它详细描述了如何从路网中的一个点移动到另一个点，包括道路的连接关系、方向、长度等信息。

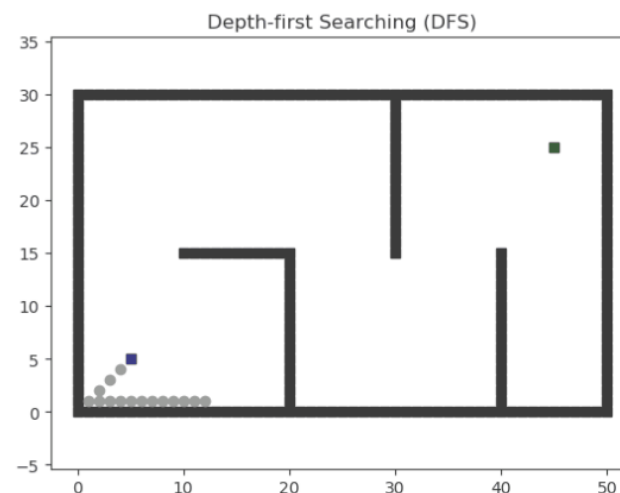
计算路径：通过调用全局路线规划器的`trace\_route`方法，计算从起始路点到结束路点的最佳路径。这个方法返回的是一系列的`(Waypoint, RoadOption)`元组，表示整个路径。使用的是[A*算法](#)。

self._graph: 这是构建的图本身，包含了路网的所有节点和边。在路径规划中，算法会在这个图上运行，以找到从起点到终点的最短路径或最优路径。

self._id_map: 这个映射表包含了图中节点的ID和它们在CARLA世界中对应的对象（如Waypoint）。这允许规划器将图算法找到的路径转换为实际的路点，这些路点可以被用来导航和控制车辆。以字典的形式展示，key是每一个entry/exit waypoint的坐标位置，value是它们在graph里的id。

self._road_id_to_edge: 这是一个从CARLA道路ID到图中边的映射。在CARLA中，每段道路都有一个唯一的ID，这个映射允许将这些道路ID与图中的边相对应。这对于理解图中的边如何对应于实际道路网络很有用。一个三层字典，最外层是carla里的道路id,中间一层是carla里的在该道路上的section id,最后一层是carla里的lane id。Road包含Section，Section包含Lane。

```
# global_route_planner.py
self._graph, self._id_map, self._road_id_to_edge = self._build_graph()
```



自动驾驶之心



automatic_control中的算法解析-局部路径规划代码解析

全局路线规划好以后，agent会初始化局部路线规划器，方便后面的轨迹规划

```
# inside set_destination() function in behavior_agent.py
self._local_planner.set_global_plan(route_trace, clean)
```

具体代码如右图所示，**waypoint buffer**里存的导航点是carla里短期跟随的路线，它每经过一个点就会把历史数据扔掉，而这种操作deque/queue数据类型效率最高（FIFO）。当**waypoint buffer**里的点都被扔出去变空之后，**waypoints_queue**就会再次弹出自己的五个点，送给**waypoint buffer**。如此循环，随着离目的地越来越近，**waypoints_queue**会被一点点pop，最终到达目的地时全部清空。

作业，学习文档中的五种行为规划：1）正常驾驶的行为规划；2）针对交通信号灯的行为规划；3）针对行人的行为规划；4）针对路上其它车辆的行为规划；5）交叉口的行为规划

```
def set_global_plan(self, current_plan):
    """
    Resets the waypoint queue and buffer to match the new plan. Also
    sets the global_plan flag to avoid creating more waypoints
    :param current_plan: list of (carla.Waypoint, RoadOption)
    :return:
    """

    # Reset the queue
    self._waypoints_queue.clear() #清空当前的路径队列
    for elem in current_plan:
        self._waypoints_queue.append(elem) #将新的全局计划中的每个元素（每个waypoint及其对应的
        road option）加入到这个队列中
    self._target_road_option = RoadOption.LANEFOLLOW #默认情况下，如果没有其他指示，车辆将继续沿
    着当前车道行驶

    # and the buffer
    self._waypoint_buffer.clear() #清空当前的waypoint缓冲区
    for _ in range(self._buffer_size): #从更新后的路径队列中取出前_buffer_size个元素填充到缓冲区
    中。这个缓冲区用于存储即将到来的waypoints，以便局部规划器可以提前做出决策。
        if self._waypoints_queue:
            self._waypoint_buffer.append(
                self._waypoints_queue.popleft())
        else:
            break

    self._global_plan = True #表示已经加载了一个全局规划，局部规划器应该遵循这个计划进行导航
```

